

1 Benchmark Architecture and Reward Design

1.1 Overall Architecture

We use a Docker-based sandbox environment. For each run, the model is given a task prompt, a source video, and tool descriptions. The model does not receive private verifier files, hidden ground truth, or other agents’ submissions.

The sandbox includes a CPU-based video and audio editing stack. These tools support media probing, trimming, concatenation, transcoding, frame extraction, audio processing, scene analysis, transcription, image analysis, schema validation, and report generation. See Appendix A for the detailed package list.

After completing the task, the agent submits:

- `submit/output.mp4`: the final edited video.
- `submit/edit_decision.json`: the edit decision record.
- `submit/run_history.md`: the action history.
- `submit/agent_transcript.md`: the agent transcript or transcript summary.

1.2 Verifier and Reward Structure

The verifier is divided into a general verifier and a task-specific verifier. The final reward is:

$$R = 0.2r_{\text{general}} + 0.8r_{\text{task}}.$$

r_{general} checks task-independent requirements such as valid output format, playable media, audio/video presence, audio quality, and whether the required submission files exist.

r_{task} checks the actual task objective. When the LLM judge is included in the task reward, we use:

$$r_{\text{task}} = 0.85r_{\text{hard_task}} + 0.15r_{\text{llm}}.$$

$r_{\text{hard_task}}$ is computed by explicit hard verifiers. r_{llm} is an LLM-as-judge score for parts that are difficult to fully verify with deterministic checks. In some experiments, the LLM judge is diagnostic only and is reported separately.

Each reward component follows an explicit rubric. Since there are many criteria and each individual weight can be small, we add hard caps: if the agent makes a severe mistake on an important criterion, the total reward is capped. This prevents major failures from being diluted by many minor passing checks.

1.3 Reward Design Principles for Video Editing

Designing rewards for video editing is difficult because a good edited video is defined by many heterogeneous standards. A final video can fail because of format, timing, audio quality, visual quality, subtitle readability, synchronization, pacing, continuity, content coverage, or many other small issues. These criteria are often local, tedious, and task-dependent. One of the main challenges is not only implementing verifiers, but first identifying the right set of standards and composing them into a reward that reflects real editing quality.

Our design borrows from coding-agent evaluation. In coding tasks, many rewards can be understood as unit tests: each test checks one expected behavior, and the final score aggregates the

results. We apply the same idea to video editing. Each rubric is treated like a media-specific unit test. For example, one test checks whether the output is a valid video file, another checks whether black frames were removed, another checks whether audio/video sync improved, another checks whether required content spans remain covered, and another checks whether subtitles are readable and not intrusive.

The second challenge is reward hacking. When a reward measures one narrow property, the agent may satisfy that property in a way that harms the final video. For example, if a reward only checks whether subtitles are readable, an agent could make subtitles huge and cover the entire frame. To reduce this, every reward should be paired with anti-reward-hacking checks. In the subtitle example, the verifier should check not only text readability, but also subtitle size, placement, duration, overlap with faces or important objects, and whether the subtitle blocks the visual content. In general, we try to anticipate how a model could exploit each metric and add counter-checks that make the hack costly.

The third challenge is subjectivity. Some possible criteria, such as whether a video is “interesting” or “funny”, are highly subjective and can vary across judges. We avoid using overly subjective rubrics as primary reward terms. Instead, we focus on properties that most reasonable evaluators would agree on: whether the video preserves the required content, whether the timeline is coherent, whether the edit removes obvious unusable sections, whether audio and video are synchronized, whether subtitles align with speech, whether the output is watchable, and whether the final result follows the task instructions.

1.4 Anti-Reward-Hacking Mechanisms

We use several mechanisms to make the reward harder to exploit:

1. **Hard caps.** If an agent makes a severe mistake on a critical requirement, the total reward is capped. This prevents a model from passing many small checks while failing the core task.
2. **Counter-checks for each metric.** For every reward component, we ask how a model could hack the metric. We then add checks for those failure modes. For example, subtitle readability must be paired with checks for excessive subtitle size, face overlap, visual occlusion, timing mismatch, and unnecessary text.
3. **Multiple reward sources.** We combine hard verifiers with LLM-based judging when appropriate. Hard verifiers provide stable objective checks; LLM judges provide coverage for high-level coherence and perceived edit quality. For subjective or model-sensitive judgments, we can sample multiple LLM judges and average their scores to reduce variance.
4. **Content preservation checks.** The verifier should punish outputs that solve artifacts by deleting too much useful content. For example, removing a damaged section is good, but deleting entire required explanation spans is not. Content preservation, source-time monotonicity, duration bounds, and required-span coverage help prevent over-trimming.

2 Details of Benchmarks

2.1 Piecewise A/V Sync Repair

This task asks the agent to repair a damaged 16:9 tutorial clip. The source video contains multiple technical defects: piecewise audio/video desynchronization, black padding, a visual flash, a

frozen/dead-air splice, a repeated bad insert, and a local audio-level problem. The agent must preserve the useful tutorial content in the original order, remove unusable sections, repair timing issues, normalize audio, and export a clean publishable video with the required format.

The task is designed to require approximately 15 genuinely sequential decisions and about 28–34 tool calls, with 30 tool calls as the expected typical solution length. A good solution should follow the dependency chain: read constraints, inspect metadata, sample frames and audio, detect boundary/artifact/sync issues, choose local repair ranges, choose an output timeline, render the output, and validate the final submission. A single command without observation-dependent diagnosis is not considered a long-horizon solution.

The targeted capability mix is:

- **Reasoning over a plan:** the agent must decide which issues are real, which ranges should be removed, which content must be preserved, and how local repairs interact.
- **Tool use:** the agent must use media inspection, audio analysis, frame sampling, timeline construction, rendering, and validation tools.
- **Multimodal grounding:** the agent must connect visual events, audio events, timestamps, and content spans.

The task does not primarily target self-correction from intermediate observations, VLM endpoint use, NLE scripting, or generative video/audio.

Injected issue	Location / magnitude	Expected repair	Why it creates horizon
Piecewise A/V desync	Four clean-time regions with offsets: +720 ms, -650 ms, +480 ms, -380 ms	Estimate and repair timing locally, not with one global offset	Requires segment-level diagnosis and local timeline decisions
Black leader and tail	1.2 s at the beginning and 0.8 s at the end	Remove boundary padding without deleting useful content	Requires distinguishing padding from content
White flash artifact	0.7 s flash around the early tutorial section	Remove the flash and keep neighboring explanation content	Requires visual artifact localization and clean splice choice
Freeze/dead-air splice	0.75 s freeze near the middle of the clip	Remove the frozen/dead-air section while preserving continuity	Requires joint visual and audio inspection
Duplicate bad insert	Repeated 2.2 s segment near the late section	Remove the duplicate without over-trimming the original content	Requires content matching and over-deletion avoidance
Local audio-level damage	A 24 s region attenuated by -7 dB	Normalize/repair speech loudness without clipping or replacing audio	Requires audio-quality analysis and export validation
Required content spans	Opening clap demo, middle explanation, final explanation	Preserve all required spans in order	Prevents the agent from solving the task by deleting too much

Table 1: Injected issues in the Piecewise A/V Sync Repair task.

2.1.1 Reward Rubric

The reward uses dense graded subscores when partial progress is meaningful, but adds hard caps for severe failures. This is a trade-off for RL signal density: dense intermediate rewards reduce variance and help distinguish partially correct edits, but they also create more opportunities for reward hacking. We therefore pair each dense reward with counter-checks or caps.

Format compliance. This dimension belongs to r_{general} and is verified by hard checks. It checks output existence, metadata readability, stream presence, duration gate, aspect ratio, codec family, and non-degenerate media. Its anti-reward-hacking role is to prevent invalid or degenerate files from receiving meaningful reward. We classify it as **Hard-to-hack**. The reward shape is mostly binary gates plus graded format subscores, which gives early dense feedback about invalid-media failures.

Audio quality. This dimension belongs to r_{general} and is verified by hard checks. It penalizes silence, clipping, bad loudness, and degenerate audio, so the agent cannot pass by muting the video or replacing the audio with silence. We classify it as **Hackable but bounded**, because audio metrics can be partially gamed, but mute, clipping, and silence hacks are detectable. The reward shape is graded because audio repair quality is continuous and a dense score provides useful RL signal.

Artifact completeness and submission files. This dimension belongs to r_{general} and is verified by hard checks. It requires the expected submission artifacts and schema fields, preventing video-only submissions with no edit evidence. We classify it as **Hard-to-hack**. The reward shape is a binary/vector set of subscores, which is low variance and easy to verify.

Provenance and logs. This dimension belongs to r_{general} and is verified by hard checks over the submitted documentation. It requires action history and transcript files, making it harder to hide arbitrary or implausible transformations. We classify it as **Hackable but bounded**, because logs can be fabricated, but missing or shallow evidence is detectable and this component has limited weight. The reward shape is graded documentation quality; it is useful as weak evidence, not as a primary task signal.

Segmented A/V sync. This dimension belongs to r_{task} and is verified by hard checks. It compares the output against private clean-reference sync anchors that the agent cannot synthesize, and evaluates each temporal region separately. We classify it as **Hard-to-hack**. The reward shape is a vector of segment subscores aggregated into a graded score. This gives dense signal for local sync repair and avoids treating a video with one repaired segment and three broken segments as fully correct.

Global A/V sync. This dimension belongs to r_{task} and is verified by hard checks. It uses private anchor matching and residual statistics across the output, preventing the agent from repairing only one local region while breaking the rest of the video. We classify it as **Hard-to-hack**. The reward shape is a graded residual score with median and p90 diagnostics, so partial timing improvement receives credit while large residuals are still penalized.

Visual artifact cleanup. This dimension belongs to r_{task} and is verified by hard checks. It checks whether black frames, white flashes, frozen frames, and related visual artifacts remain in the output. The anti-reward-hacking design is to combine a graded cleanup score with hard caps when obvious artifacts remain. We classify it as **Hackable but bounded**, because visual heuristics can be gamed, but major artifacts are detectable and capped. The reward shape is graded plus capped: dense enough to reward partial cleanup, but bounded by artifact gates.

Content preservation. This dimension belongs to r_{task} and is verified by hard checks. It uses private required spans, source-time monotonicity, and duration bounds so the agent cannot win by deleting difficult content. We classify it as **Hard-to-hack**. The reward shape is a vector of span coverage scores plus a graded aggregate, which gives dense credit for partial preservation while still punishing over-trimming.

Duplicate or bad insert cleanup. This dimension belongs to r_{task} and is verified by hard checks. It rewards overlap with the known duplicate region while penalizing over-removal and non-monotonic source timelines. We classify it as **Hard-to-hack**, because the duplicate location is private and the agent cannot simply synthesize a passing timeline. The reward shape is graded, balancing successful removal against the risk of deleting valid content.

Audio cleanup. This dimension belongs to r_{task} and is verified by hard checks. It combines audio quality and dead-air checks, preventing mute/pad hacks and clipping-heavy normalization. We classify it as **Hackable but bounded**, because audio processing can be gamed locally, but silence, clipping, and dead-air failures are measurable. The reward shape is graded because audio cleanup is continuous and dense reward is preferable.

Black boundary cleanup. This dimension belongs to r_{task} and is verified by hard checks. It checks both the start and end boundaries so the model cannot add padding to satisfy duration constraints. We classify it as **Hard-to-hack**. The reward shape is mostly binary/graded boundary scoring, which is simple and robust.

LLM edit quality judge. This dimension is either part of r_{task} as a minority term or reported as a diagnostic score. It uses an LLM judge for aspects such as continuity and perceived polish that are difficult to fully capture with deterministic checks. The anti-reward-hacking design is to keep this score as a minority term or diagnostic only, so the hard verifier remains dominant. We classify it as **Soft / vibes**. The reward shape is graded and useful as a tiebreaker, not as the primary reward signal.

Hard caps. Hard caps apply to the final reward and combine multiple verifier signals. They cap the total score for missing output, invalid media, bad duration, low content preservation, low task score, remaining major artifacts, or severe cleanup failures. We classify this mechanism as **Hard-to-hack**. The reward shape is binary at the cap level: sparse, but necessary to prevent severe failures from being diluted by many minor passing checks.

All rubric dimensions in this section can be included in the final training signal, with weights adjusted for the specific training setup; the same dimensions can also be exposed as denser, more step-level rewards.

Model	Coding agent	r_{task}	r_{general}	Total reward
Claude Sonnet 4.6	Claude Code	0.641	0.822	0.677
Gemini 3.1 Pro Preview ¹	Gemini CLI	0.612	0.717	0.633
GPT-5.4 Mini	Codex	0.548	0.832	0.605
Claude Opus 4.7	Claude Code	0.507	0.919	0.589
GPT-5.5	Codex	0.499	0.882	0.576
GPT-5.4	Codex	0.446	0.837	0.525
Claude Haiku 4.5	Claude Code	0.432	0.832	0.512
Gemini 3.1 Flash-Lite	Gemini CLI	0.399	0.762	0.471

Table 2: Model results on the Piecewise A/V Sync Repair task. Scores are reported after verifier execution with the final reward formula and hard caps.

The results suggest that the strongest runs were not simply the ones with the best general media compliance. Most models produced playable outputs with reasonable r_{general} , but their total rewards differed mainly through r_{task} , especially local A/V sync repair, visual artifact cleanup, and content preservation. Claude Sonnet 4.6 achieved the best overall score because it combined relatively strong task repair with valid output formatting. Gemini 3.1 Pro Preview also performed well in the carried successful run, while several other models preserved content and format but left more task-specific defects unresolved.

2.2 Expert Pancake Tutorial Extraction

This task asks the agent to edit a landscape “50 People Try” pancake challenge video into a 55–65 second vertical social tutorial. The source contains many non-target regions, including novice attempts, bloopers, jokes, reaction material, and end cards. The agent must find the expert or proficient pancake-making portion, preserve the cooking actions in a coherent order, reframe the landscape footage into a real portrait crop, add short burned-in step captions, keep useful original audio, and export a publishable 9:16 H.264/AAC video.

The task is designed to be a moderate long-horizon editing problem rather than a one-command transform. The expected horizon is 6 genuinely sequential decisions and roughly 9–15 tool calls, with 12 tool calls as the typical solution length. A good solution should follow the dependency chain: inspect metadata and visual samples, identify the expert tutorial region, map the cooking actions, choose source ranges and portrait crop strategy, render the vertical short, then verify and patch duration, framing, captions, audio, and content coverage. A single `ffmpeg` command without observation-dependent source selection and verification is not considered a successful long-horizon solution.

The targeted capability mix is:

- **Multimodal grounding:** the agent must distinguish expert cooking footage from visually similar novice or blooper material, and must align captions with visible cooking actions.
- **Reasoning over a plan:** the agent must decide which source ranges form a compact step-by-step tutorial, how to preserve order, and how to avoid unrelated filler.

The task does not primarily target A/V sync repair, multi-source editing, object removal, super-resolution, frame interpolation, complex motion graphics, or generative video/audio.

For this benchmark, the final score uses:

$$r_{\text{hard}} = 0.20r_{\text{general}} + 0.80r_{\text{task}}, \quad R_{\text{before caps}} = 0.85r_{\text{hard}} + 0.15r_{\text{llm}}.$$

The reported reward is then the minimum of $R_{\text{before caps}}$ and any active hard cap. The LLM judge is used as a minority semantic signal for caption/action consistency and mobile tutorial quality, while the hard verifier remains dominant.

2.2.1 Reward Rubric

The reward uses dense graded subscores for source selection, step coverage, captions, and reframing, but applies hard caps for severe failures. This gives useful RL signal for partial progress while preventing obvious hacks such as fake vertical padding, deleting hard sections, using external footage, or adding captions that do not match the visible action.

Format compliance. This dimension belongs to r_{general} and is verified by hard checks. It requires playable output, fprobe-readable metadata, 9:16 aspect ratio, duration gate, codec family, stream presence, and non-degenerate media. We classify it as **Hard-to-hack**. The reward shape is binary gates plus graded format subscores, so invalid artifacts are capped before small positives can dominate.

Audio quality. This dimension belongs to r_{general} and is verified by hard checks. It penalizes silence, clipping, bad loudness, missing audio, and silent padding so the agent cannot pass by muting or replacing the original audio. We classify it as **Hackable but bounded**, because audio metrics can be partially gamed, but mute, clipping, and silence hacks are detectable. The reward shape is graded because audio quality is continuous and dense signal is useful.

Video quality. This dimension belongs to r_{general} and is verified by hard checks. It checks black, blank, frozen, and visually degenerate output artifacts, as well as basic visual integrity. We classify it as **Hackable but bounded**, because visual heuristics can be gamed locally, but degenerate videos receive low media-integrity scores even if they satisfy metadata. The reward shape is graded with gates.

Source authenticity and timeline. This dimension belongs to r_{general} and is verified by hard checks. It matches sampled output frames back to the provided source and checks source-time diversity and monotonicity. This discourages external footage, generated visuals, still images, and arbitrary padding. We classify it as **Hard-to-hack**. The reward shape is a vector of frame-match subscores plus a graded aggregate, which gives dense credit for valid source use while detecting major provenance failures.

Provenance and logs. This dimension belongs to r_{general} and is verified by hard checks over submitted artifacts. It requires valid `edit_decision.json`, source-range consistency, `run_history.md`, and `agent_transcript.md`. We classify it as **Hackable but bounded**, because logs can be fabricated, but missing or shallow evidence is detectable and this component has limited weight. The reward shape is binary and graded documentation subscores, cross-checked against media evidence.

Tutorial segment purity. This dimension belongs to r_{task} and is verified by hard checks. It uses private allowed and negative source intervals to penalize novice attempts, bloopers, end cards, and off-tutorial leakage. We classify it as **Hard-to-hack**. The reward shape is a graded impurity score with hard caps; heavy non-tutorial leakage caps the final reward.

Temporal order. This dimension belongs to r_{task} and is verified by hard checks. It uses source-time monotonicity and first-hit order of required cooking actions, preventing shuffled clips or captions that describe a different order than the footage. We classify it as **Hard-to-hack**. The reward shape is graded because partial ordering progress is meaningful, but severe order failures can still trigger caps.

Visual step completeness. This dimension belongs to r_{task} and is verified by hard checks. It measures coverage of private required visual intervals for pan prep, batter, bubble cue, flip, and plating. We classify it as **Hard-to-hack**. The reward shape is a vector of step subscores plus a graded aggregate; the minimum step score and hard caps prevent passing by showing only one easy action repeatedly.

True vertical reframe. This dimension belongs to r_{task} and is verified by hard checks. It combines aspect ratio, black-border checks, blurred-padding checks, resolution, and ROI containment. We classify it as **Hackable but bounded**, because crop heuristics can be gamed, but fake vertical layouts with a landscape video inside padding are detectable and capped. The reward shape is graded with caps.

Step caption completeness. This dimension belongs to both r_{task} and r_{llm} , and is verified by combined hard and LLM checks. Hard checks reward visible sequential captions, caption timing metadata, and step-keyword coverage; the LLM judge checks whether the captions form coherent tutorial steps. We classify it as **Hackable but bounded**. The reward shape combines a hard keyword/metadata score with an LLM semantic score, and oversized, missing, or semantically weak captions can trigger caps.

Caption/visual alignment. This dimension belongs to both r_{task} and r_{llm} , and is verified by combined hard and LLM checks. Hard checks use caption cue timing and source matches when available; the LLM judge inspects sampled frames around caption transitions to decide whether each caption matches the visible action. We classify it as **Soft / vibes**, because semantic alignment is difficult to make fully deterministic. The reward shape is graded and supported by hard timing evidence, but it remains a minority term.

LLM tutorial quality judge. This dimension belongs to r_{llm} and is verified by an LLM judge. GPT-5.5 judges step caption completeness, caption/action alignment, mobile readability, tutorial naturalness, and prompt fit. We classify it as **Soft / vibes**. The reward shape is a vector of graded subscores, weighted at 15% before caps, so it can break ties and catch semantic failures but cannot rescue a major hard-verifier failure.

Hard caps. Hard caps apply to the final reward and combine hard and LLM signals. They cap missing output, invalid media, bad duration, wrong aspect or resolution family, fake vertical format, heavy non-tutorial leakage, external or unmatched material, weak visual step coverage, severe order failure, missing captions, weak step captions, and severe LLM-detected caption/action failures. We classify this mechanism as **Hard-to-hack**. The reward shape is binary at the cap level: sparse, but necessary to prevent severe failures from being diluted by many minor passing checks.

The reward intentionally uses a vector of dense subscores rather than a single binary pass/fail. This gives a usable RL signal for partial progress: a run can receive credit for finding the right expert region even if its captions are weak, or for producing a valid vertical output even if one

cooking step is missing. The trade-off is a larger reward-hacking surface. We therefore pair dense scores with private source intervals, source-frame matching, ROI checks, caption-layout checks, and hard caps. The design treats subjective LLM judgments as a minority semantic layer, while the main reward remains grounded in media evidence that the agent cannot easily synthesize.

All rubric dimensions in this section can be included in the final training signal, with weights adjusted for the specific training setup; the same dimensions can also be exposed as denser, more step-level rewards.

Model	Coding agent	r_{task}	r_{general}	Total reward
GPT-5.4	Codex	0.961	0.948	0.939
GPT-5.4 Mini	Codex	0.879	0.881	0.865
Claude Opus 4.7	Claude Code	0.809	0.928	0.808
GPT-5.5	Codex	0.796	0.887	0.805
Claude Sonnet 4.6	Claude Code	0.749	0.896	0.767
Gemini 3.1 Flash-Lite	Gemini CLI	0.712	0.892	0.550
Claude Haiku 4.5	Claude Code	0.265	0.834	0.366
Gemini 2.5 Pro	Gemini CLI	0.651	0.897	0.350

Table 3: Model results on the Expert Pancake Tutorial Extraction task. Scores use GPT-5.5 as the LLM judge through OpenRouter and are reported after the final reward formula and hard caps.

The results show that this task separates general media compliance from task-specific editing quality. Most runs produced playable videos with strong r_{general} , but the final ranking was driven by expert-region selection, visual step coverage, true vertical reframing, and caption/action alignment. GPT-5.4 and GPT-5.4 Mini scored highest because they avoided hard caps and achieved strong task coverage. The lower-scoring runs illustrate the intended cap behavior: Gemini 3.1 Flash-Lite was capped for weak semantic captions and alignment, Claude Haiku 4.5 was capped for non-tutorial leakage, and Gemini 2.5 Pro was capped for fake or weak vertical reframing despite otherwise reasonable general media scores.

2.3 Rough Interview Cleanup and Captioning

This task asks the agent to turn a rough talking-head interview clip into a clean 60–75 second excerpt. The final video should preserve a clear, self-contained explanation about helping young people stay motivated and thrive at work. The agent must remove long dead air, obvious false starts, and repeated phrase loops while keeping the speaker’s meaning and argument order intact. It must also normalize speech audio, add readable burned-in captions for all spoken words, save the timing as `submit/captions.srt`, and avoid black frames, frozen frames, unrelated footage, background music, synthetic narration, and heavy decorative overlays.

The task is designed for roughly 10 genuinely sequential decisions and about 18–22 tool calls, with 20 tool calls as the expected typical solution length. A good solution should follow the dependency chain: inspect metadata and constraints, transcribe or sample speech, identify the coherent explanation span, detect dead air and repeated speech, plan a source-preserving timeline, render captions and audio normalization, validate duration and format, inspect caption readability, then patch any duration, caption, or audio problems. A single `ffmpeg` trim without transcript grounding and post-render verification is not considered a successful long-horizon solution.

The targeted capability mix is:

- **Reasoning over a plan:** the agent must decide which spoken content forms a complete answer, which hesitations are safe to remove, and how to avoid changing the argument.

- **Tool use:** the agent must use metadata probes, ASR or transcript extraction, audio analysis, timeline construction, caption rendering, normalization, and final validation.
- **Self-correction from intermediate observations:** the task expects at least one verify-and-patch pass for duration, caption coverage, audio levels, and visual/caption layout.

The task does not primarily target object removal, super-resolution, frame interpolation, lip sync generation, complex motion graphics, generative narration, or cinematic color grading.

For this benchmark, the final reward uses the shared benchmark structure:

$$R = 0.2r_{\text{general}} + 0.8r_{\text{task}}, \quad r_{\text{task}} = 0.85r_{\text{hard_task}} + 0.15r_{\text{llm}}.$$

r_{general} covers task-independent media validity, format compliance, audio integrity, visual degeneracy checks, and required artifacts. $r_{\text{hard_task}}$ covers transcript-grounded speech cleanup, semantic anchor preservation, caption timing/text consistency, spoken-audio normalization, source fidelity, and anti-reward-hacking checks. r_{llm} is a GPT-5.5 judge used for meaning preservation, edit naturalness, caption visual layout, caption text/sync quality, and publishability.

2.3.1 Reward Rubric

The reward uses a vector of dense subscores because partial progress is meaningful: an agent can preserve the right explanation but have weak caption layout, or clean dead air well while still missing some semantic anchors. Dense rewards reduce RL variance and make these differences visible. The trade-off is a larger reward-hacking surface, so dense terms are paired with private transcript anchors, ASR checks, source-fidelity checks, duration gates, caption-layout caps, and LLM quality caps.

Format and media integrity. This dimension belongs to r_{general} and is verified by hard checks. It requires a playable output, valid metadata, H.264/AAC-style streams, 1280x720 format, duration gate, audio/video presence, and non-degenerate frames. Its anti-reward-hacking role is to prevent invalid files, still images, blank video, or metadata-only outputs from receiving meaningful reward. We classify it as **Hard-to-hack**. The reward shape is mostly binary gates plus graded format subscores, which gives early dense feedback about invalid-media failures while still using hard gates for severe cases.

General audio quality. This dimension belongs to r_{general} and is verified by hard checks. It penalizes silence, clipping, missing audio, bad loudness, and degenerate audio, so the agent cannot pass by muting the video or replacing the original speech with silence. We classify it as **Hackable but bounded**, because loudness metrics can be partially gamed, but mute, clipping, and silence hacks are directly detectable. The reward shape is graded because audio quality is continuous and dense reward is useful for RL.

Required artifacts and logs. This dimension belongs to r_{general} and is verified by hard checks over the submitted files. It requires `output.mp4`, `captions.srt`, `edit_decision.json`, `run_history.md`, and `agent_transcript.md`, and cross-checks edit decisions against media evidence when possible. We classify it as **Hackable but bounded**, because logs can be fabricated, but missing, malformed, or shallow evidence is detectable and this component has limited weight. The reward shape is a vector of binary and graded documentation subscores, giving weak process signal without dominating final video quality.

Speech cleanup. This dimension belongs to r_{task} and is verified by hard checks. It measures long dead-air removal, spoken-word density, repeated n-gram rate, and cleanup of obvious false-start-like loops. The anti-reward-hacking design is to pair cleanup with duration and content-preservation checks, so the agent cannot obtain a high cleanup score by deleting most of the speech. We classify it as **Hackable but bounded**. The reward shape is graded because cleanup quality is continuous: partial dead-air removal should receive credit, but over-trimming is capped by preservation and duration gates.

Semantic anchor preservation. This dimension belongs to r_{task} and is verified by hard and combined checks. It uses private source transcript anchors and ASR output matching to verify that the final edit preserves the key explanation about motivation, young people, and thriving at work. The LLM judge also checks whether the explanation remains self-contained. We classify it as **Hard-to-hack**, because the private anchors are not given to the agent and cannot be satisfied by arbitrary captions or generic motivational language. The reward shape is a vector of anchor-recall and timing-aware token scores, giving dense credit for partial preservation while preventing direct targeting of hidden spans.

Meaning and order preservation. This dimension belongs to r_{task} through the LLM component and is verified by combined evidence. GPT-5.5 compares the source and output transcripts for self-contained meaning, argument order, and whether edits change the speaker’s point. ASR anchors, duration gates, and source-fidelity checks bound deletion and reordering hacks. We classify it as **Soft / vibes**, because semantic order cannot be fully reduced to deterministic media tests. The reward shape is a graded LLM subscore used as a minority term, useful as a tiebreaker and semantic guardrail rather than the primary reward signal.

Caption coverage and timing. This dimension belongs to r_{task} and is verified by hard and combined checks. It requires `submit/captions.srt`, checks SRT/ASR token F1, timing-aware token F1, cue coverage, and caption-speech consistency, while the LLM judge checks whether the captions accurately reflect the spoken content. We classify it as **Hackable but bounded**: captions can be gamed by dumping text or using generic summaries, but ASR matching, timing checks, and visual layout checks detect the main hacks. The reward shape is a dense vector of token and timing subscores, which provides local RL signal for caption repair.

Caption visual layout and subject occlusion. This dimension belongs to r_{task} and also appears in hard-cap logic. It uses an LLM judge to assess whether burned-in captions are readable without covering the speaker’s face or main subject. Severe occlusion, oversized subtitles, or heavy decorative overlays trigger caps, preventing the common hack of making subtitles huge to improve OCR/readability. We classify it as **Soft / vibes**, because face occlusion and readability are partly perceptual, but the cap makes severe failures bounded. The reward shape is a graded LLM layout score plus hard caps for obvious visual failures.

Spoken-audio normalization. This dimension belongs to r_{task} and is verified by hard checks. It checks speech loudness, clipping, pumping proxies, and non-silent output while requiring the original spoken content to remain present. We classify it as **Hackable but bounded**, because audio normalization can be optimized narrowly, but silence, clipping, and content-removal hacks are measurable. The reward shape is graded because comfortable speech loudness is continuous and benefits from dense feedback.

Source fidelity and visual cleanliness. This dimension belongs to r_{task} and hard-cap logic, and is verified by hard and combined checks. It checks for black frames, frozen frames, unrelated footage, synthetic narration, heavy overlays, and source-format naturalness. The LLM publishability score catches visible failures not fully covered by deterministic metrics. We classify it as **Hackable but bounded**, because visual heuristics can be gamed locally, but major non-source or degenerate outputs are detectable and capped. The reward shape is graded with caps, so partial visual cleanup receives credit but severe artifacts cannot be hidden by other positives.

Hard caps. Hard caps apply to the final reward and combine multiple verifier signals. They cap the total score for missing output, invalid media, duration failure, weak semantic preservation, weak caption layout, low LLM quality, severe occlusion, or obvious non-source/synthetic content. We classify this mechanism as **Hard-to-hack**. The reward shape is binary at the cap level: sparse, but necessary to prevent severe failures from being diluted by many minor passing checks.

All rubric dimensions in this section can be included in the final training signal, with weights adjusted for the specific training setup; the same dimensions can also be exposed as denser, more step-level rewards.

Model	Coding agent	r_{task}	r_{general}	Total reward
Gemini 3.1 Pro Preview ²	Gemini CLI	0.838	0.831	0.837
GPT-5.5	Codex	0.787	0.816	0.792
GPT-5.4	Codex	0.798	0.745	0.787
Claude Sonnet 4.6	Claude Code	0.728	0.785	0.739
Claude Opus 4.7	Claude Code	0.664	0.824	0.696
GPT-5.4 Mini	Codex	0.667	0.809	0.696
Gemini 3.1 Flash-Lite	Gemini CLI	0.630	0.733	0.651
Claude Haiku 4.5	Claude Code	0.645	0.796	0.550

Table 4: Model results on the Rough Interview Cleanup and Captioning task. Scores use GPT-5.5 as the LLM judge and are reported after the final reward formula and hard caps.

The results show that this task distinguishes transcript-grounded editing from generic media compliance. Gemini 3.1 Pro Preview scored highest because it combined strong semantic anchor preservation, caption quality, and audio normalization. GPT-5.5 and GPT-5.4 were close but failed different parts of the rubric: GPT-5.5 had stronger general media quality but weaker semantic-anchor and LLM caption-layout scores, while GPT-5.4 had stronger task preservation but lower general media compliance. The lower-scoring runs illustrate why the reward mixes hard and LLM checks: several outputs were playable and had acceptable audio, but lost important explanation anchors, produced weak caption layouts, or triggered quality caps for semantic or visual issues.

3 Metric for Data Selection

To check whether the selected benchmark tasks measure different abilities, we use the reward profile across models as a task-level fingerprint. Let $v_t \in \mathbb{R}^8$ be the vector of final rewards for task t , using the model order

(GPT-5.5, GPT-5.4, GPT-5.4 Mini, Claude Opus,
Claude Sonnet, Claude Haiku, Gemini Pro, Gemini Flash).

²Gemini 3.1 Pro Preview returned a nonzero runner status because a later API request hit quota after the submission artifacts had already been written. The verifier scored the submitted artifacts successfully.

From the tables above, the three reward vectors are

$$\begin{aligned} v_{\text{sync}} &= (0.576, 0.525, 0.605, 0.589, 0.677, 0.512, 0.633, 0.471), \\ v_{\text{pancake}} &= (0.805, 0.939, 0.865, 0.808, 0.767, 0.366, 0.350, 0.550), \\ v_{\text{interview}} &= (0.792, 0.787, 0.696, 0.696, 0.739, 0.550, 0.837, 0.651). \end{aligned}$$

We center each vector by subtracting its mean, $\tilde{v}_t = v_t - \bar{v}_t \mathbf{1}$, and define the pairwise task similarity and distance as

$$\rho(t_i, t_j) = \frac{\tilde{v}_{t_i}^\top \tilde{v}_{t_j}}{\|\tilde{v}_{t_i}\|_2 \|\tilde{v}_{t_j}\|_2}, \quad d(t_i, t_j) = \frac{1}{\pi} \arccos(\rho(t_i, t_j)).$$

The overall dispersion score is the average pairwise angular distance:

$$D = \frac{2}{T(T-1)} \sum_{i < j} d(t_i, t_j), \quad T = 3.$$

For the current three tasks, the pairwise correlations are

$$\rho_{\text{sync,pancake}} = 0.162, \quad \rho_{\text{sync,interview}} = 0.513, \quad \rho_{\text{pancake,interview}} = 0.303,$$

with corresponding normalized angular distances

$$d_{\text{sync,pancake}} = 0.448, \quad d_{\text{sync,interview}} = 0.328, \quad d_{\text{pancake,interview}} = 0.402.$$

Thus the average dispersion is

$$D = 0.393.$$

This suggests that the three tasks are not redundant: they rank the same models differently and therefore likely emphasize different capabilities. The sync and interview tasks are the closest pair, but their correlation is still only moderate; the pancake task is more separated from the other two. This metric is useful for data selection because it does not require hand-labeling every latent capability. If two tasks had nearly identical reward vectors, they would add little new training signal; if a task has a separated reward vector with nontrivial score variance, it is more likely to contribute distinct supervision.

A CPU Sandbox Tool and Package List

Command-line tools.

- ffmpeg
- ffprobe
- bash
- python >= 3.11

Python packages.

- numpy
- scipy
- opencv-python
- moviepy
- av
- librosa
- soundfile
- pydub
- scenedetect
- faster-whisper
- Pillow
- scikit-image
- pandas
- pydantic
- tqdm
- ultralytics
- colour-science
- noisereduce
- imageio
- imageio-ffmpeg
- matplotlib
- jsonschema